



AI TEAM TRAINING '27

TASK 6 PHASE II

Task 1.1 Modernizing the Machine Translation Model

In the workshop, we built a machine translation model using frequency-based pre-embedding methods, an attention mechanism, and a bidirectional LSTM. The goal of this task is to improve the existing approach by incorporating modern word-embedding techniques and evaluating the model using appropriate machine translation metrics.

Technical requirements

- Fork the existing [notebook](#) from Kaggle and use it as the starting point for the project.
- Replace the existing frequency-based representation with a modern word-embedding method, such as:
 - Word2Vec
 - GloVe
 - FastText
- Modify and adapt the notebook to work with the selected word-embedding approach while maintaining the machine translation architecture.
- Evaluate the translation model using appropriate metrics, particularly:
 - BLEU (Bilingual Evaluation Understudy)
 - ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- Create a README file that:
 - Describes the overall approach.
 - Explains the selected word-embedding method.
 - Explains the BiLSTM and attention mechanism.
 - Describes the preprocessing and training process.
 - Explains the evaluation metrics (BLEU and ROUGE).
 - Reports and discusses the obtained results.
 - Highlights any additional techniques used to improve model performance.

Task 1.2: From Notebook to Production.

Throughout our training, we have completed several practical tasks to strengthen our AI and machine learning skills. However, most of these tasks have focused primarily on individual AI concepts rather than building a complete, production-oriented project that also develops our software engineering skills.

In this task, we will develop a production-oriented Image Caption Generator that automatically generates a natural-language description for a given image. The project will use the Flickr8k Dataset, which contains 8,000 images, with each image paired with five different human-written captions.

The project will combine Computer Vision and Natural Language Processing (NLP) techniques. A deep learning model will be developed to extract meaningful visual features from images and generate appropriate textual descriptions. In addition to model development, the project will focus on software engineering practices such as modular design, clean code, project structure, testing, documentation, and deployment.

Objectives:

The main objectives of this project are to:

1. Build an end-to-end Image Caption Generation system.
2. Apply deep learning techniques to combine Computer Vision and NLP.
3. Learn how to work with and preprocess a real-world image-caption dataset.
4. Develop an image feature extraction pipeline using a pretrained CNN/Transfer Learning.
5. Build a sequence-generation model for generating image captions.
6. Evaluate the generated captions using appropriate metrics such as BLEU, ROUGE, and/or METEOR.
7. Apply software engineering best practices when developing an AI project.
8. Organize the project into reusable and maintainable modules rather than relying on a single notebook.
9. Implement testing and validation to improve the reliability of the system.
10. Package and deploy the trained model as a usable application or API.

Technical requirements

1. Dataset

- Use the Flickr8k Dataset.
- Each image should be associated with its corresponding five reference captions.
- Implement appropriate preprocessing for both images and captions.
- Split the dataset into training, validation, and test sets while avoiding data leakage.

2. Image Feature Extraction

- Use a pretrained computer vision model such as ResNet, EfficientNet, or InceptionV3 to extract image features.
- Use transfer learning rather than training the visual feature extractor entirely from scratch.
- Store or cache extracted features where appropriate to improve training efficiency.

3. Text Processing

Implement a complete caption-processing pipeline including:

- Tokenization.
- Vocabulary construction.
- Special tokens such as <start> and <end>.
- Padding/sequencing.
- Handling unknown and rare words.
- Converting captions into numerical representations.

4. Caption Generation Model

Develop a deep learning architecture that combines image features with text information. For example:

CNN/Pretrained Vision Model → Image Features → LSTM/GRU → Generated Caption

5. Model Training

- Implement an appropriate training pipeline.
- Use techniques such as early stopping, learning-rate scheduling, and model checkpointing where appropriate.
- Track training and validation performance.
- Save the best-performing model.

6. Evaluation

Evaluate the model on unseen test images using appropriate image-captioning metrics, such as:

- BLEU
- ROUGE
- METEOR

In addition to numerical evaluation, provide qualitative examples showing:

Input Image → Generated Caption → Reference Captions

7. Software Engineering

The project should not be implemented as a single large notebook. Organize the code into reusable components.

8. Production/Deployment

Create a simple interface through which a user can upload an image and receive a generated caption.

Possible implementation options include:

- FastAPI for an API-based solution.
- Streamlit or Gradio for a simple user interface.
- Dockerization

9. Documentation

Create a comprehensive README.md explaining:

- Project overview.
- Dataset.
- Architecture.
- Preprocessing.
- Training process.
- Evaluation metrics and results.
- How to install and run the project.
- How to use the application/API.
- Example inputs and outputs (e.g. Record Video and put it in README).

10. Model Storage and Sharing

- Save the trained model and any required model artifacts in a dedicated cloud storage location such as HuggingFace.
- The project should include a public/shareable link to the model files in the README.md.